Architecture

Standalone Mail → Telegram App Architecture

Цель

Собрать **единое самостоятельное приложение**, которое: - не зависит от Hermes agent runtime; - не использует внутренние Hermes memory/cron/tool abstractions; - можно перенести на другой VPS/сервер как обычный проект; - работает как отдельный продукт для приема email и доставки в Telegram.

Принцип

Hermes можно использовать только как временную среду разработки/администрирования. Сам продукт должен запускаться **сам по себе**.

Границы standalone-приложения

Внутрь приложения входят: 1. Gmail ingestion - чтение входящих писем через Gmail API; - получение raw/full message; - извлечение alias из `To`, `Delivered-To`, `X-Original-To`, `Received` при необходимости. 2. Alias registry - генерация случайных alias; - хранение связи `alias -> telegram_chat_id`; - статус alias (active/disabled). 3. Message processing - нормализация письма; - выделение subject/from/to/date/body/snippets; - дедупликация по message-id / gmail-message-id. 4. Telegram delivery - отправка в Telegram Bot API; - fallback на plain text, если HTML не прошел; - логирование delivery status. 5. Persistence - локальная БД приложения; - таблицы aliases, inbound_messages, deliveries, app_settings. 6. Config/runtime - `.env` / config file; - отдельные OAuth credentials; - отдельный bot token; - собственный systemd unit / Docker container. 7. Optional AI layer - отдельный модуль summary/extract/Q&A; - должен быть отключаемым и не влиять на базовую доставку писем.

Что НЕ должно быть зависимостью

Нельзя завязывать продукт на: - Hermes memory; - Hermes cronjob; - Hermes Telegram transport; - основной Hermes Google auth; - runtime-файлы главного агента; - внутренние tool wrappers.

Допустимо использовать Hermes только для: - первичной разработки; - ручного администрирования; - миграции/деплоя.

Рекомендуемый стек

Минимальный и переносимый вариант: - Python 3.11+ - FastAPI — API/health/admin endpoints - SQLAlchemy + SQLite initially - Alembic — миграции БД - httpx — Telegram Bot API / внешние API - Google API client или простой REST-клиент для Gmail API - systemd или Docker Compose для запуска

Почему SQLite сначала

Для первой версии это лучший компромисс: - один файл; - легко мигрировать; - не нужен отдельный PostgreSQL; - можно потом перейти на PostgreSQL через SQLAlchemy/Alembic.

Планируемая структура проекта

```
/opt/hermes/mail-telegram-app/
  app/
    __init__.py
    main.py                  # FastAPI entrypoint
    config.py                # settings from env
    db.py                    # engine/session/base
    models/
      alias.py
      inbound_message.py
      delivery.py
      app_setting.py
    services/
      gmail_client.py        # Gmail API polling/fetch
      alias_service.py       # alias generation + lookup
      parser_service.py      # header/body parsing
      telegram_client.py     # Telegram Bot API send
      routing_service.py     # alias -> chat routing
      ingestion_service.py   # poll -> parse -> store -> deliver
    repositories/
      aliases.py
      inbound_messages.py
      deliveries.py
    api/
      health.py
      aliases.py
      admin.py
    schemas/
      alias.py
      message.py
    utils/
      logging.py
      html_to_text.py
  migrations/
  tests/
    test_alias_service.py
    test_parser_service.py
    test_routing_service.py
```

```
    test_ingestion_service.py
  .env.example
  requirements.txt
  alembic.ini
  Dockerfile
  docker-compose.yml
  systemd/
    mail-telegram-app.service
  README.md
  ARCHITECTURE.md
```

Базовая модель данных

aliases

- id
- alias_local_part
- alias_email
- telegram_chat_id
- telegram_user_id nullable
- description nullable
- is_active
- created_at
- updated_at

inbound_messages

- id
- gmail_message_id
- internet_message_id
- thread_id
- alias_email
- delivered_to
- from_address
- subject
- received_at
- body_text
- body_html nullable
- raw_headers_json

- parse_status
- created_at

deliveries

- id
- inbound_message_id
- telegram_chat_id
- telegram_message_id nullable
- status
- error_text nullable
- delivered_at nullable
- created_at

app_settings

- id
- key
- value
- updated_at

Режим работы v1

Простой poller внутри приложения: 1. по таймеру читает Gmail API; 2. берет новые письма; 3. парсит alias из заголовков; 4. ищет alias в БД; 5. сохраняет письмо; 6. отправляет в Telegram; 7. пишет статус доставки.

Это лучше, чем завязка на Hermes cron, потому что: - переносимо; - прозрачно; - управляется самим приложением; - не зависит от конкретного агентного окружения.

Режим запуска

Нужно поддержать 3 режима: 1. `api` — HTTP API + healthcheck; 2. `worker` — poller/ingestion loop; 3. `all-in-one` — для простого single-host запуска.

Пример: - `python -m app.main --mode all-in-one` или - отдельные контейнеры `api` и `worker`.

Миграция на другой сервер

Чтобы приложение было легко перенести, на новом сервере должно быть достаточно: 1. скопировать репозиторий; 2. перенести `.env`; 3. перенести OAuth

credentials/token; 4. перенести SQLite DB или поднять PostgreSQL; 5. запустить Docker Compose или systemd unit.

То есть миграция должна быть **операцией уровня приложения**, а не восстановления среды Hermes.

Конфиг приложения

Переменные должны быть такими, чтобы не зависеть от Hermes: - `APP_ENV` - `APP_BASE_URL` - `DATABASE_URL` - `GMAIL_CLIENT_SECRET_PATH` - `GMAIL_TOKEN_PATH` - `GMAIL_POLL_SECONDS` - `TELEGRAM_BOT_TOKEN` - `DEFAULT_FROM_ALIAS_DOMAIN` - `LOG_LEVEL` - `AI_PROVIDER` - `AI_API_KEY`

Внешние интеграции

Приложение должно иметь только эти внешние зависимости: - Gmail API - Telegram Bot API - доменная почта/Cloudflare routing - опционально AI provider

Что делаем дальше

1. Создаем каркас standalone-приложения.
2. Поднимаем SQLite + Alembic.
3. Реализуем alias registry.
4. Реализуем Gmail poller.
5. Реализуем Telegram delivery.
6. Делаем end-to-end тест.
7. Добавляем Docker/systemd упаковку.

Решение

Архитектурное решение зафиксировано: **строим один standalone mail-telegram-app как обычный переносимый backend-сервис, а не функцию Hermes agent.**

Bootstrap Plan

Standalone mail-telegram-app bootstrap plan

1. Create Python package layout.
2. Add failing tests for config loading.
3. Add failing tests for alias generation and recipient extraction.
4. Implement minimal code to pass tests.
5. Add app entrypoint and health endpoint.
6. Add persistence skeleton and models.
7. Add env example, requirements, README.
8. Run tests.